

EXAM POV NOTES (Q&A FORMAT)

Q1: What are two core benefits of traditional relational databases? (2 Marks)

- **Persistent Data Storage:** Relational databases are excellent at storing large amounts of data permanently, organizing it for fast and easy retrieval by applications, which is essential as main memory is volatile.
- **Concurrency Control:** They manage situations where many users or systems access and modify data simultaneously, using transactions to coordinate interactions, prevent conflicts (like double-booking), and allow for error rollback.

Q2: Explain the value and key features that made relational databases dominant for decades.

(5 Marks)

- **Long-Standing Dominance:** For many decades, relational databases were the absolute foundation and often the only viable choice for building serious applications, proving their robustness and reliability.
- **Reliable Persistent Data:** They ensure data is kept safe and permanently accessible by storing it in a persistent "backing store" (like a disk), overcoming the limitations of volatile main memory.
- **Robust Concurrency Management:** Relational databases manage simultaneous data access and modification by multiple users or systems through ACID transactions, which contain the complexity of concurrent operations and allow for changes to be undone if errors occur.
- **Seamless Application Integration:** They facilitate collaboration among multiple applications, often developed by different teams, by allowing them to share and interact with a consistent set of data stored in a single database.
- **Standardized Model and Language:** Relational databases provide a (mostly) standard model and a universal language, SQL, enabling developers to learn a consistent approach that applies across many projects and different database vendors.
- **ACID Transaction Guarantees:** They offer **ACID (Atomicity, Consistency, Isolation, Durability) transactions**, which guarantee that a set of operations either fully completes as a single unit or none of it happens at all, ensuring data integrity.

Q3: Discuss the historical dominance and the fundamental benefits provided by relational databases that made them the "king" of data storage. (10 Marks)

- **Decades of Rule:** Relational databases, primarily accessed via SQL, were the undisputed king for decades, serving as the default and often only choice for serious application development due to their robust nature.
- **Core Business Problem Solvers:** They expertly addressed fundamental business needs: **ensuring data safety, managing concurrent access for large user bases**, providing a unified data source for multiple teams, and offering a standard developer language (SQL).
- **Reliable Persistent Storage:** A key benefit is their ability to store and manage vast amounts of **persistent data**. They effectively move data from volatile main memory to durable backing stores, organizing it so applications can quickly retrieve specific pieces of information.
- **Sophisticated Concurrency Control:** They **manage the complexities of concurrent access from many users or systems** by enforcing control through transactions. This prevents common errors like double-booking and allows for error handling through transaction rollbacks.
- **Facilitating Application Integration:** Relational databases are crucial for enterprise environments where multiple applications, often from diverse teams, need to collaborate. A shared database allows applications to easily use each other's data, with the database's concurrency control managing access effectively.

- **Standardized Query Language (SQL):** The adoption of SQL provided a largely consistent and **standardized model**. This meant developers and database professionals could apply their skills and knowledge across various projects and different database products with similar core mechanisms.
- **ACID Transaction Guarantees:** They offered ACID (**A**tomicity, **C**onsistency, **I**solation, **D**urability) transactions, a rock-solid guarantee that a series of operations is treated as an indivisible unit. This ensured data integrity, where operations either fully succeed or completely fail, and changes are instantly visible.
- **Predictable and Structured Data:** The relational model inherently organizes data into well-defined tables with rows and columns and predefined schemas, which is excellent for structured data where relationships are clearly established using foreign keys.
- **Mature Ecosystem:** Over their long reign, a comprehensive ecosystem of tools, established best practices, and skilled professionals grew around relational databases, further cementing their reliability and widespread use.
- **Foundational Stability:** They provided a **solid, dependable foundation** for enterprise computing, underpinning critical operations with their guarantees of data safety and consistency.

Q4: Identify two major problems that contributed to the emergence of NoSQL databases. (2 Marks)

- **Impedance Mismatch:** This refers to the fundamental clash between how application code structures data (as complex objects) and how relational databases store it (as flat tables), necessitating frustrating and complex translation.
- **Scale Challenges:** Relational databases faced significant difficulties efficiently handling the immense growth in data volumes and user traffic that exploded with the internet, particularly when scaling out across many machines.

Q5: Explain the "Impedance Mismatch" problem experienced by developers working with relational databases. (5 Marks)

- **Fundamental Clash:** Impedance mismatch describes the basic conflict between the rich, object-oriented data structures preferred by application code and the rigid, tabular data model enforced by relational databases.
- **Application's Object View:** In their code, developers work with intuitive, complex objects, such as a 'Customer' object that naturally contains a list of 'Orders', with each 'Order' holding 'Products'.
- **Relational Database's Flat View:** When saving this data, relational databases require this beautiful structure to be "smashed flat," spreading its components across multiple, separate tables with simple rows and columns.
- **Constant Translation Overhead:** This continuous process of flattening objects for storage and then rebuilding them from fragmented table rows for application use is not only annoying but also slow and introduces significant complexity.
- **Object-Relational Mapping (ORM) Challenges:** While ORM tools aim to automate this translation, they often introduce their own complexities, leading to error-prone code and performance bottlenecks, especially with intricate queries or large datasets.
- **Expressiveness Gap:** SQL, as a declarative language, has a different way of expressing operations compared to imperative programming languages, making it challenging to bridge the gap between how data is processed in code versus the database.

Q6: Discuss the twin problems of "Impedance Mismatch" and "Scale" that presented significant challenges for traditional relational databases, ultimately leading to the NoSQL movement.

(10 Marks)

- **Impedance Mismatch** – The Developer's Frustration: This problem, long a source of frustration for application developers, highlights the fundamental clash between the rich, hierarchical object models used in programming languages and the flat, tabular structure of relational databases.
- **Object-Oriented Data Representation**: In code, developers naturally model complex entities as objects, where a 'customer' object might encapsulate 'orders,' which in turn contain 'products,' representing a cohesive unit.
- **Relational Flattening**: To store these objects, relational databases require them to be "smashed flat," distributing parts of the logical object across multiple tables (e.g., customer details in one table, order details in another, product details in a third).
- **Inefficient Translation**: This constant, laborious process of serializing (flattening) objects for storage and deserializing (rebuilding) them for application use is slow, complex, and a major source of development overhead, contributing to performance concerns and error-prone code.
- **The Scale Problem** – Internet Data Explosion: The second critical issue was the sheer, unprecedented scale of data generated by the internet. Websites began tracking every click, view, and social connection from millions of users, leading to "insane" volumes of data.
- **Resource Demands**: Coping with this massive increase in data and user traffic demanded significantly more computing resources than traditional systems could easily provide.
- **"Scale Up" Limitations**: The traditional relational approach to increasing capacity was "scaling up" – buying bigger, more powerful, and incredibly expensive servers. However, this approach had inherent limits to growth and became prohibitively costly.
- **"Scale Out" Necessity**: The alternative, "scaling out," involved linking many smaller, cheaper, commodity computers together in a cluster. This proved to be more cost-effective and resilient, as the failure of one cheap machine didn't cripple the entire system.
- **Relational Database's Struggle with Scale Out**: Relational databases, designed for single, powerful machines and strong ACID consistency across the entire dataset, struggled to adapt efficiently to distributed "scale-out" architectures, particularly regarding maintaining consistency.
- **The "Perfect Storm"**: These two intense pressures—developer frustration over impedance mismatch and the massive technical need for cost-effective horizontal scaling—created the "perfect storm" that compelled the industry to seek entirely new ways of handling data, giving birth to the NoSQL movement.

Q7: What does "NoSQL" truly stand for, and when did the modern movement begin?

(2 Marks)

- **"Not Only SQL"**: The term NoSQL is often misunderstood; it actually stands for "Not Only SQL," indicating a need for a broader range of database tools beyond just relational databases, not a complete rejection of SQL.
- **Modern Movement Origin**: The modern NoSQL movement, as we understand it today, largely originated at a developer meetup in San Francisco in 2009, bringing together engineers working on new open-source, distributed, non-relational database technologies.

Q7: Explain the core concepts behind the emergence and definition of NoSQL databases. (5 Marks)

- **Response to Relational Challenges:** NoSQL databases emerged directly to address the limitations of traditional relational databases, specifically the impedance mismatch and the inability to efficiently scale out horizontally to meet internet-level demands.
- **"Not Only SQL" Philosophy:** The name "NoSQL" signifies a shift to a "bigger toolbox" of data solutions, implying that relational databases are one tool among many, rather than the sole option.
- **Fundamental Shift to Aggregates:** At its heart, NoSQL represents a fundamental change in how data is conceptualized, moving away from splitting information into disparate tables towards grouping related data together into cohesive units called aggregates.
- **Aggregate Data Model:** An aggregate is a collection of related data treated as a single logical unit, like a customer and their orders bundled together, which inherently matches how application code often models objects.
- **Resolves Impedance Mismatch:** This aggregate model directly resolves the impedance mismatch, as the data's storage structure aligns with the application's object model, eliminating the complex "smashing and rebuilding" process.
- **Designed for Horizontal Scale:** Many NoSQL databases are purpose-built to scale horizontally across clusters of cheaper, commodity machines, providing the flexibility and resilience needed for massive data volumes and high traffic, which was a significant challenge for relational systems.

Q8: Discuss the key drivers behind the emergence of NoSQL, its true meaning, and its impact on modern data persistence strategies. (10 Marks)

- **Impedance Mismatch Frustration:** A primary driver was the persistent frustration among developers regarding the impedance mismatch, the slow and complex translation required between an application's object-oriented data structures and a relational database's flat table model.
- **Internet Scale Demands:** The explosion of data and user traffic on the internet created an urgent need for systems that could scale horizontally across distributed clusters of inexpensive hardware, a challenge that traditional relational databases were not inherently designed to meet efficiently.
- **Formalization at a Meetup:** While the term "NoSQL" appeared earlier ironically for a relational database, its modern usage stems from a San Francisco meetup in 2009, where engineers from companies like Cassandra and MongoDB gathered to name their new category of "open-source, distributed, nonrelational databases".
- **"Not Only SQL" Principle:** The true meaning of NoSQL is "Not Only SQL," signaling an end to the "one database to rule them all" mindset. It champions a broader "toolbox" of database solutions, acknowledging that SQL databases are still valuable but not universally suitable.
- **Aggregate-Oriented Data Design:** A core conceptual shift introduced by NoSQL is the adoption of the aggregate data model. This involves grouping logically related data into single, self-contained units (aggregates) that are treated as one, mirroring how applications typically view objects.
- **Solving Impedance Mismatch:** This aggregate approach directly addresses the impedance mismatch by storing data in a way that matches the application's native object structure, drastically reducing the complexity and overhead of data translation.
- **Polyglot Persistence Paradigm:** The NoSQL movement gave rise to polyglot persistence, a strategy where different types of databases are used for different specific data storage

tasks within the same application ecosystem. This means choosing the right tool based on the data's nature and application requirements.

- **Emphasis on Flexibility and Scale:** NoSQL databases are characterised by their incredible schema flexibility (schema-less) and their inherent ability to scale out massively on commodity hardware, offering significant advantages for agile development and handling large, dynamic datasets.
- **Trade-offs in Consistency and Querying:** While offering flexibility and scale, NoSQL systems often involve trade-offs, such as a lack of a standardised query language (each system has its own API) and a move from immediate ACID consistency to eventual consistency for distributed operations.
- **A Mature Mindset Shift:** The lasting legacy of NoSQL is a more mature approach to data management. The focus shifted from a "SQL vs. NoSQL war" to asking, "What does my data look like, and what does my app need to do with it?"—leading to intelligent selection from a diverse database landscape.

Q9: State two key characteristics of NoSQL databases. (2 Marks)

- **Schema-less/Flexible Schema:** NoSQL databases typically do not require a predefined schema, allowing each record to have different fields and making them highly adaptable to evolving data structures.
- **Horizontal Scalability:** They are fundamentally designed to scale by adding more servers to a distributed system, efficiently handling increasing data volumes and high user traffic.

Q10: Enumerate six advantages of using NoSQL databases. (5 Marks)

- **Flexible Schema:** NoSQL databases are often schema-less or schema-flexible, which allows for easier adaptation to changing data requirements, particularly beneficial when dealing with unstructured or semi-structured data.
- **Horizontal Scalability:** Many NoSQL databases are engineered for horizontal scalability, meaning they can handle growing amounts of data and traffic by simply adding more servers to a distributed system.
- **High Performance:** They are optimised for specific use cases, providing high performance for certain types of queries and operations, and are efficient at managing large volumes of read and write operations for low-latency data access.
- **Variety of Data Models:** NoSQL databases support a diverse range of data models including document-oriented, key-value, column-family, and graph databases, enabling organisations to select the most appropriate model for their specific application needs.
- **Agile Development and Iteration:** Their inherent flexibility makes them highly suitable for agile development methodologies, allowing developers to quickly iterate on applications without being constrained by rigid schema definitions.
- **Cost-effectiveness:** NoSQL databases can be more cost-effective in certain scenarios, especially for large-scale distributed systems, as they often leverage commodity hardware and scale horizontally, potentially reducing infrastructure costs.

Q11: Discuss the main characteristics, advantages, and disadvantages of NoSQL databases. (10 Marks)

- **Schema-less or Schema-flexible:** A defining characteristic is their ability to operate without a rigid, predefined schema. This means each record can have a different set of fields, making it easy to accommodate non-uniform or evolving data without complex alterations.
- **Non-relational Nature:** Unlike traditional SQL databases, NoSQL systems do not adhere to the fixed tabular structure of rows and columns, offering alternative data models better suited for complex or less structured data patterns.

- **Horizontal Scalability:** They are specifically designed for horizontal scaling, allowing systems to handle increasing data loads and traffic by distributing data and operations across numerous commodity servers in a cluster, which is more cost-effective than scaling up.
- **Distributed Architecture:** Many NoSQL databases are built with a distributed architecture across multiple nodes, inherently providing improved performance by parallelising operations and enhancing fault tolerance by replicating data across the cluster.
- **High Performance:** NoSQL databases are often optimised for specific types of queries and workloads, allowing them to deliver high-performance read and write operations, crucial for applications requiring low-latency access to large datasets.
- **Variety of Data Models:** NoSQL is a broad category encompassing various data models, such as document stores (e.g., MongoDB), key-value stores (e.g., Redis), column-family stores (e.g., Apache Cassandra), and graph databases (e.g., Neo4j), offering specialised solutions for different data types and access patterns.
- **BASE (Basically Available, Soft State, Eventually Consistent):** Many NoSQL databases follow the BASE model, which prioritises availability and partition tolerance over strict, immediate consistency. This means the system remains operational during network partitions but may temporarily provide inconsistent data, which eventually reconciles.
- **Disadvantage - Lack of Standardization:** A significant drawback is the absence of a universal, standardised query language like SQL. Each NoSQL database typically has its own unique API and query language, which can increase the learning curve and make it challenging to switch between different systems.
- **Disadvantage - Limited ACID Transactions:** NoSQL databases often trade strict ACID (Atomicity, Consistency, Isolation, Durability) transaction guarantees for performance and scalability. While atomicity is usually maintained within a single aggregate, coordinating ACID-level consistency across multiple aggregates typically becomes the responsibility of the application developer.
- **Disadvantage - Limited Support for Complex Queries:** While excellent for specific, often simple, data access patterns, NoSQL databases may lack the robust capabilities for complex queries, joins, and ad-hoc analytics that relational databases offer, which can be a limitation for intricate data retrieval and reporting needs.

Q12: Name two scenarios where NoSQL databases are particularly useful. (2 Marks)

- **Flexible Schema Requirements:** NoSQL databases are ideal when data is unstructured, semi-structured, or subject to frequent changes, as their schema-less nature allows for easy adaptation.
- **Scalability and High Throughput:** They are well-suited for applications that need to handle massive amounts of data and high user traffic due to their design for horizontal scaling across distributed systems.

Q13: Explain the concept of "polyglot persistence" and provide scenarios where NoSQL databases are preferred. (5 Marks)

- **Polyglot Persistence Defined:** This approach advocates for using different types of databases for different specific jobs within the same application, effectively choosing the "right tool for the job" from a diverse database "toolbox".
- **Tailored Solutions:** It acknowledges that no single database technology is optimal for all data types and access patterns, promoting the selection of databases that best match the specific requirements of each part of an application.
- **Scenario 1: Flexible Schema Needs:** NoSQL databases are highly preferred when dealing with unstructured or semi-structured data, or when the data model is expected to evolve frequently, thanks to their schema-less or schema-flexible nature.

- **Scenario 2: Massive Scale and High Throughput:** They are ideal for applications requiring horizontal scalability to manage rapidly growing data volumes and extremely high user traffic, as NoSQL databases are designed for distributed architectures and efficient read/write operations.
- **Scenario 3: Rapid Development & Agile Iterations:** The flexibility of NoSQL databases aligns perfectly with agile development methodologies, allowing developers to iterate and deploy changes quickly without the constraints of rigid schema modifications.
- **Example Use Case:** An application might use a traditional relational database for critical financial transactions requiring strict ACID guarantees, while simultaneously employing a flexible and fast NoSQL database for managing a user's shopping cart or activity feed.

Q14: Elaborate on the "polyglot persistence" approach and provide a comprehensive overview of when to choose NoSQL databases over traditional relational ones. (10 Marks)

- **Polyglot Persistence – The Modern Approach:** Polyglot persistence signifies a mature strategy in data management, advocating for the use of multiple database technologies within a single application ecosystem. The goal is to select the database that is the best fit for specific data types, access patterns, and functional requirements, rather than relying on a single, monolithic solution.
- **"Right Tool for the Job":** This concept underscores the importance of understanding the inherent nature of the data and the application's needs before selecting a data store, moving beyond the "one king to rule them all" mindset.
- **NoSQL for Flexible Schema Requirements:** NoSQL databases are the preferred choice when dealing with unstructured or semi-structured data, or when the data's schema is highly dynamic and prone to frequent changes. Their schema-less nature allows for rapid adaptation and agile development without restrictive predefined structures.
- **NoSQL for Scalability and High Throughput:** They are exceptionally well-suited for applications that demand massive horizontal scalability to handle rapidly increasing data volumes and extremely high user traffic. NoSQL databases are inherently designed for distributed architectures, leveraging commodity hardware for cost-effective expansion.
- **NoSQL for Rapid Development and Agile Iterations:** The inherent flexibility of NoSQL data models, particularly their schema flexibility, makes them a strong fit for agile development methodologies. Developers can iterate quickly on application features without being hampered by rigid schema migration processes.
- **NoSQL for Big Data Applications:** When applications need to process and store large volumes of unstructured or semi-structured data—a common characteristic of big data—NoSQL databases provide superior performance and specific optimisations compared to traditional relational systems.
- **NoSQL for High Availability and Fault Tolerance:** Many NoSQL databases are architected to be distributed across multiple nodes and often follow the BASE consistency model, prioritising high availability and partition tolerance. This makes them ideal for systems where continuous operation is paramount, even in the face of network issues or node failures.
- **NoSQL for Real-time Applications (IoT):** For scenarios like Internet of Things (IoT) where devices generate continuous streams of diverse data requiring low-latency write operations and real-time access, NoSQL databases offer the performance and flexibility needed.
- **Cost-Effectiveness at Scale:** By enabling horizontal scaling on commodity hardware, NoSQL solutions can be significantly more cost-effective for large-scale distributed systems than the expensive "scale-up" approach often required by traditional relational databases.
- **Strategic Application of Polyglot Persistence:** An example of polyglot persistence in practice would be using a robust relational database for core, high-consistency financial

transactions (leveraging ACID) while simultaneously deploying a flexible NoSQL database to manage high-volume, less critical data like user session information or a product catalogue.

Q15: What is the core concept of an "aggregate" in data modelling? (2 Marks)

- **Unit of Related Data:** An aggregate is a collection of related data that is consistently treated and manipulated as a single, indivisible unit.
- **Complex, Nested Structure:** It allows for a more complex, nested data structure, such as lists and other records within it, which closely mirrors how application code models objects.

Q16: Explain the aggregate data model and its primary advantage over the relational model. (5 Marks)

- **Data Model Definition:** A data model describes how users perceive and interact with data in a database, serving as the conceptual interface, distinct from the internal storage mechanism.
- **Aggregate Orientation:** This approach focuses on operating on data in units that possess a more complex, self-contained structure, allowing for the nesting of lists and other records within a single entity.
- **"Aggregate" as a Unit of Consistency:** Originating from Domain-Driven Design, an aggregate is defined as a collection of related objects that are treated as a single unit for data manipulation and the management of consistency.
- **Example:** An Order: In an aggregate model, an entire order—including the customer's information, shipping address, and all order items—is bundled together as one coherent document or structure, reflecting the application's view.
- **Addresses Impedance Mismatch:** The primary advantage is that this model directly resolves the impedance mismatch problem. By storing data in a format that closely aligns with the application's object model, it eliminates the need to "smash flat" and then "rebuild" data across multiple disparate tables.
- **Atomic Manipulation:** Changes typically ensure atomicity only within a single aggregate, simplifying consistency concerns for closely related data.

Q17: Discuss the aggregate data model, contrasting it with the relational model, and detailing its consequences for transactions and data integrity. (10 Marks)

- **Data Model as Perception:** A data model defines how data is perceived and interacted with by users and applications, serving as the conceptual blueprint for data organisation, distinct from its physical storage.
- **Relational Model's Atomised View:** The relational model dissects information into simple tuples (rows), which are inherently limited, preventing the nesting of records or lists of values. This simplicity underpins the relational approach but fragments logically related data.
- **Aggregate Orientation's Holistic View:** In stark contrast, aggregate orientation adopts an approach where data is treated in units possessing a complex, nested structure. This allows for data to be grouped logically, such as an entire order with its details, mirroring real-world objects.
- **Aggregate from Domain-Driven Design:** An aggregate is fundamentally a collection of related objects that are treated as a single, consistent unit for data manipulation and managing consistency boundaries. It acts as a transactional boundary.
- **Directly Solves Impedance Mismatch:** The aggregate model effectively resolves the impedance mismatch. By storing data as cohesive aggregates, the database's structure aligns closely with the application's object model, eliminating the complex and slow translation process of flattening objects into tables and then reconstructing them.

- **Relational Databases are "Aggregate-Ignorant":** Traditional relational databases are termed "aggregate-ignorant" because their model lacks an inherent concept of an aggregate. They require logically connected data to be spread across multiple tables, which can be cumbersome to reassemble.
- **Transactional Consequences - Atomicity within Aggregate:** A critical consequence is that aggregate-oriented databases typically only provide ACID (Atomic, Consistent, Isolated, Durable) transaction guarantees for operations *within a single aggregate*.
- **Transactional Consequences - Cross-Aggregate Management:** If an application needs to perform atomic manipulations across multiple aggregates, this coordination and consistency management often becomes the responsibility of the application code, unlike relational databases which natively support multi-row, multi-table ACID transactions.
- **Importance of Data Access Modelling:** When designing aggregates, it is crucial to consider how the data will be read and the potential side effects on related data that might reside outside a given aggregate. This might involve reading whole objects and parsing them client-side or splitting objects and maintaining references.
- **Trade-off: Local Consistency vs. Global Consistency:** While aggregates provide strong consistency and simplified operations for contained units, the trade-off is often a move away from immediate, global ACID consistency across the entire database, leaning towards models like eventual consistency for distributed operations.

Q18: What is a "schema-less database"? (2 Marks)

- **No Predefined Schema:** A schema-less database does not require a fixed, predefined structure (schema) to be established before data can be stored.
- **Flexible Record Structure:** It allows each individual record within the database to have a different set of fields, making it highly adaptable to non-uniform data and evolving requirements.

Q19: Discuss the concept and advantages of schema-less databases in the context of NoSQL. (5 Marks)

- **Core NoSQL Feature:** The concept of being schema-less is a common and defining characteristic across virtually all forms of NoSQL databases, differentiating them significantly from relational systems.
- **No Upfront Definition:** Unlike relational databases, where developers must first define a strict schema (tables, columns, data types) before any data can be inserted, schema-less databases remove this prerequisite entirely.
- **Handling Non-Uniform Data:** This flexibility is particularly advantageous for managing non-uniform data, where different records naturally possess different sets of fields. It avoids forcing all rows into a rigid "straightjacket" that can be awkward for diverse data types.
- **Facilitates Agile Development:** The absence of a rigid schema significantly supports agile development methodologies. Developers can quickly iterate on application changes and adapt to evolving data requirements without the time-consuming and complex process of schema migrations.
- **Easier Adaptation to Change:** It simplifies the process of adapting to changing data requirements, making it much easier to store and manage unstructured or semi-structured data, which is prevalent in modern applications.
- **Reduced Development Overhead:** By removing the need for upfront schema design and complex ORM mapping, schema-less databases can reduce development overhead and allow for faster time-to-market for certain types of applications.

Q20: What are materialized views and why are they used? (2 Marks)

- **Precomputed and Cached Query Results:** Materialized views are essentially database views whose results are computed in advance and then stored (cached) on disk, rather than being generated on-the-fly with every query.
- **Performance Optimisation for Stale Data:** They are primarily used to improve query performance for data that is read heavily but can tolerate being somewhat stale or slightly out of date, reducing the computational cost of frequent, complex queries.

Q21: Explain the role and significance of materialized views, particularly in aggregate-oriented databases. (5 Marks)

- **Performance through Precomputation:** Materialized views enhance performance by precomputing the results of complex queries and caching them on disk. This avoids the need for expensive calculations every time the data is accessed by a client.
- **Hiding Costly Computations:** They provide a mechanism to shield clients from the computational expense of certain views, making data retrieval faster and more efficient for frequently accessed but complex data.
- **Ideal for Read-Heavy, Stale Data:** Materialized views are most effective for scenarios where data is accessed very often, but strict, real-time consistency isn't always critical. Users can tolerate the data being slightly out of sync with the underlying base data.
- **NoSQL Adoption of the Term:** Although NoSQL databases generally do not have traditional SQL-style "views," they have adopted the term "materialized view" to describe their own methods for precomputed and cached query results.
- **Central for Aggregate-Oriented Systems:** Materialized views play a more central role in aggregate-oriented databases than in relational ones. This is because many application queries might not align perfectly with the predefined aggregate structures, making precomputed views a crucial optimisation for these "non-aggregate-friendly" access patterns.
- **Bridging Aggregate Gaps:** They help bridge the gap when a query needs to perform complex aggregations or join data that spans multiple distinct aggregates, providing an optimised, pre-built answer for common analytical or reporting needs.

Q23: How do graph databases store data, and what are they good for? (2 Marks)

- **Nodes and Edges:** Graph databases store data in nodes, which represent entities, and connect these nodes with edges, which represent the relationships between them.
- **Interconnection Modelling:** This model is particularly effective for storing and querying data that has many complex interconnections or relationships between entities.

Q24: What is a key consideration when modelling data aggregates regarding data access? (2 Marks)

- **Read Patterns:** When designing aggregate data models, it is essential to consider how the data will ultimately be read and accessed by the application, not just how it will be written or updated.
- **Side Effects on Related Data:** Designers must also carefully consider the potential side effects on related data that may not be contained within the immediate aggregate, as aggregate boundaries impact consistency and retrieval.

Q25: Describe the data model of graph databases and provide examples of their common use cases. (5 Marks)

- **Node and Edge Structure:** Graph databases use a data model fundamentally based on nodes and edges. Nodes represent discrete entities (like people, places, or events), while edges represent the relationships or connections between these nodes.
- **Interconnected Data Focus:** This structure is specifically designed to excel at modelling and querying data that naturally forms a network or graph, where the relationships between data points are as important as the data points themselves.
- **Aggregate-Ignorant:** Notably, similar to relational databases, graph databases are considered aggregate-ignorant, meaning their core model does not inherently group data into single, transactional aggregates.
- **Use Case: Social Networking:** A prime example is in social networking sites, where graph databases are used to store and efficiently query the complex web of relationships between users, such as friendships, follows, or shared interests.
- **Use Case: Fraud Detection:** They are highly effective in fraud detection systems, allowing analysts to uncover intricate patterns and hidden relationships between accounts, transactions, and individuals that might indicate fraudulent activity.
- **Use Case: Networks and IT Operations:** Graph data models are also widely applied in network management and IT operations to map network topologies, track dependencies between systems, or analyse access control policies, enabling quick identification of impacts or vulnerabilities.